

第六章 stm32单片机的通用定时器

- ◆TIM定时器的概念
- ◆TIM做定时器
- ◆TIM做计数器

硬件计数器模块

一、TIM概述

- TIM定时器区别于SYSTICK，它不是内核的，而是片内外设的定时器。

对比项	SYSTICK 滴答定时器	TIMx 通用定时器 (TIM1/TIM2...)
所属位置	CPU 内核内部	芯片内核外部，片上外设
来源	ARM 内核标准自带，所有 ARM 芯片通用	ST 厂商额外集成的硬件定时器模块
功能	仅简单延时、系统计时	定时中断、产生PWM波、测量外部信号频率、测量脉宽、计数外部脉冲、驱动电机
时钟来源	固定系统时钟	可选择内部时钟、外部时钟
中断	只有 1 个简单中断	多通道独立中断
独立工作	依附内核	完全独立，CPU 休眠它也能跑。 即独立于 CPU 内核工作，有自己的时钟、预分频、计数、中断、通道、捕获比较

一、TIM概述

★STM32F103RBT6芯片内部包含 4 个定时器外设：

TIM1、TIM2、TIM3、TIM4

★按照功能强弱可以分为两类：

①TIM1为高级定时器，功能最为强大；

②TIM2、TIM3、TIM4 为普通定时器，基础功能完备。

★功能：定时、PWM波形产生、输入捕获、输出比较、管脚计数等

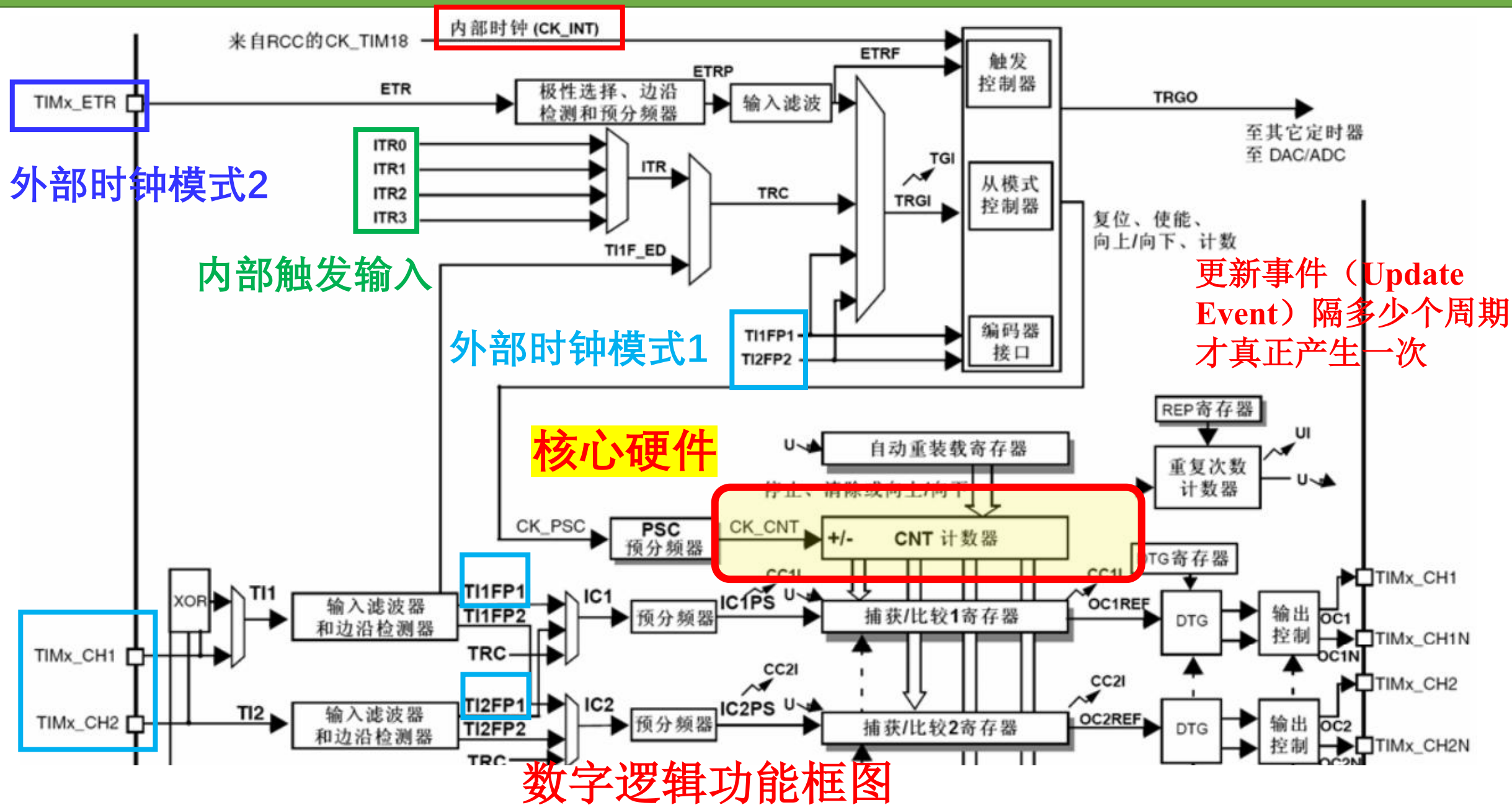
★所有功能底层核心硬件都是：

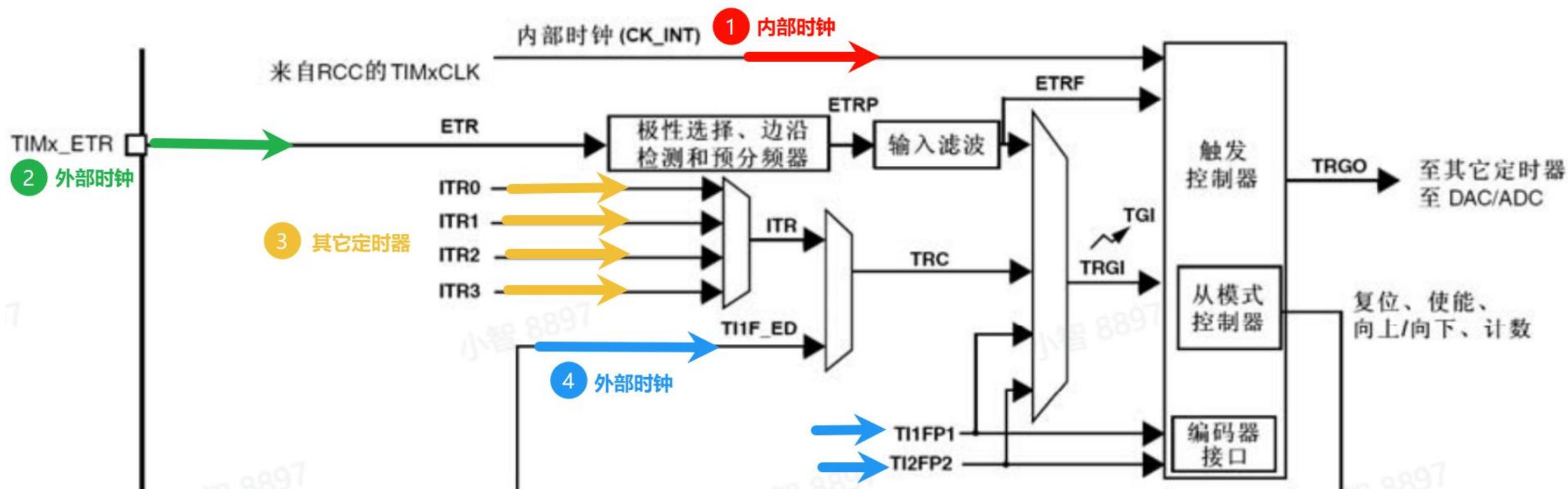
一个16位的自动重装载定时计数器。

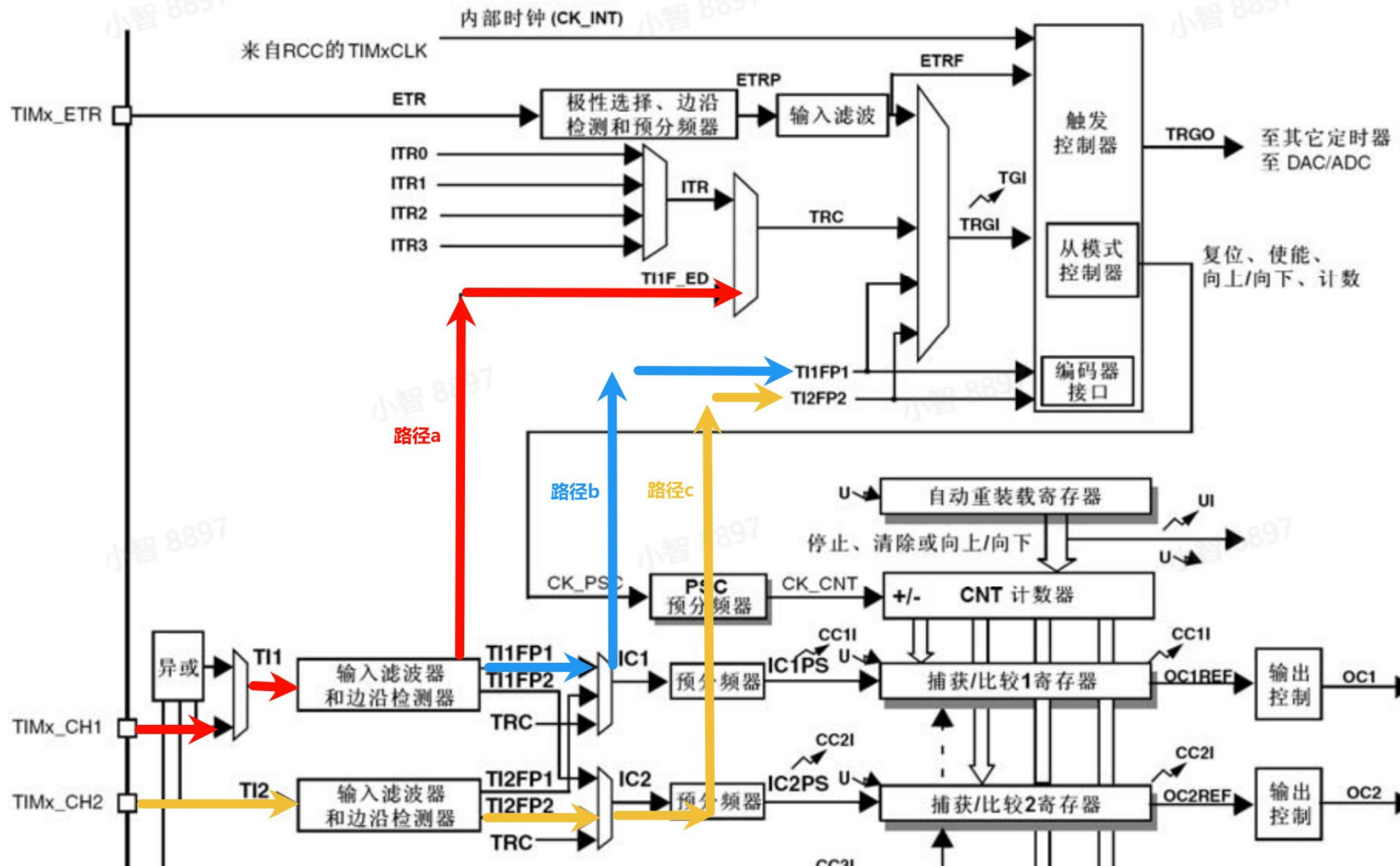
计数器时钟可由下列时钟源提供：

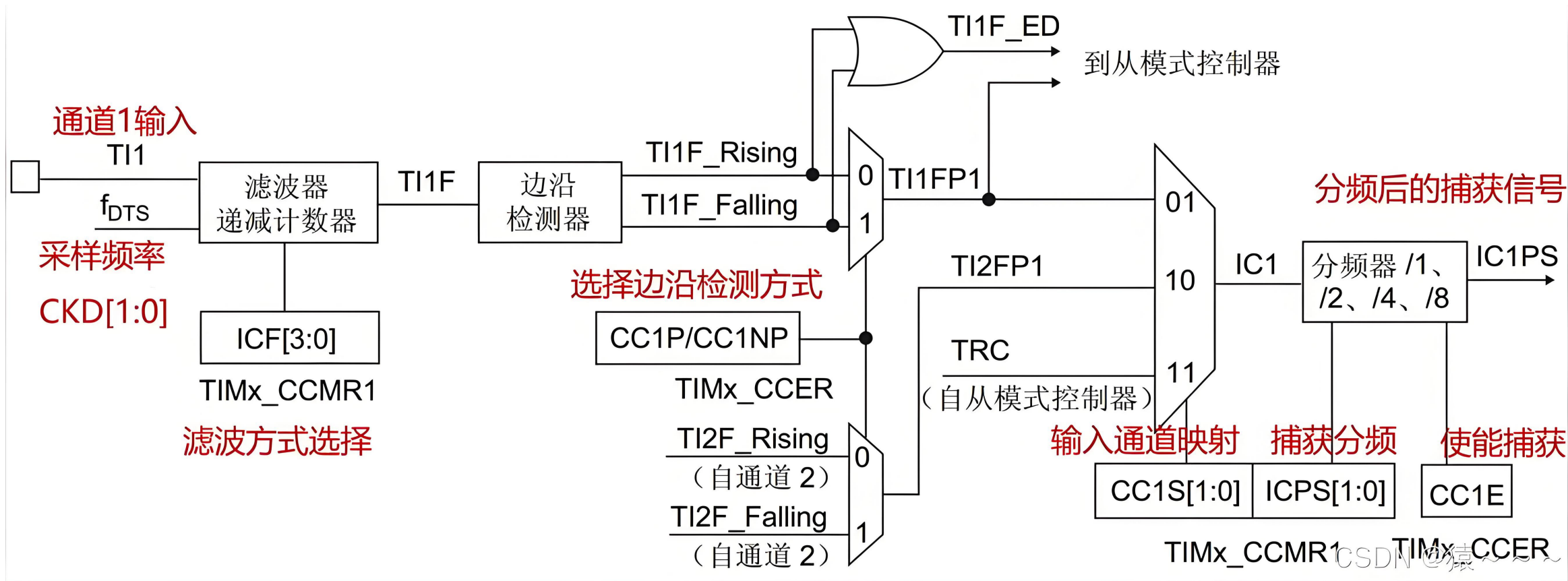
- ①内部时钟(CK_INT)：来自于定时器自己的内部基础时钟（72Mhz）
- ②外部时钟模式1：从通道输入引脚(TIx)进来的外部时钟，一般指通道1和通道2引脚信号。
- ③外部时钟模式2：从专用(ETR)External Trigger引脚进来的外部时钟，一般指可以复用为TIMx_ETR的GPIO引脚。
- ④内部触发输入(ITRx)Internal Trigger x：使用一个定时器作为另一个定时器的预分频器。

二、高级定时器TIM1

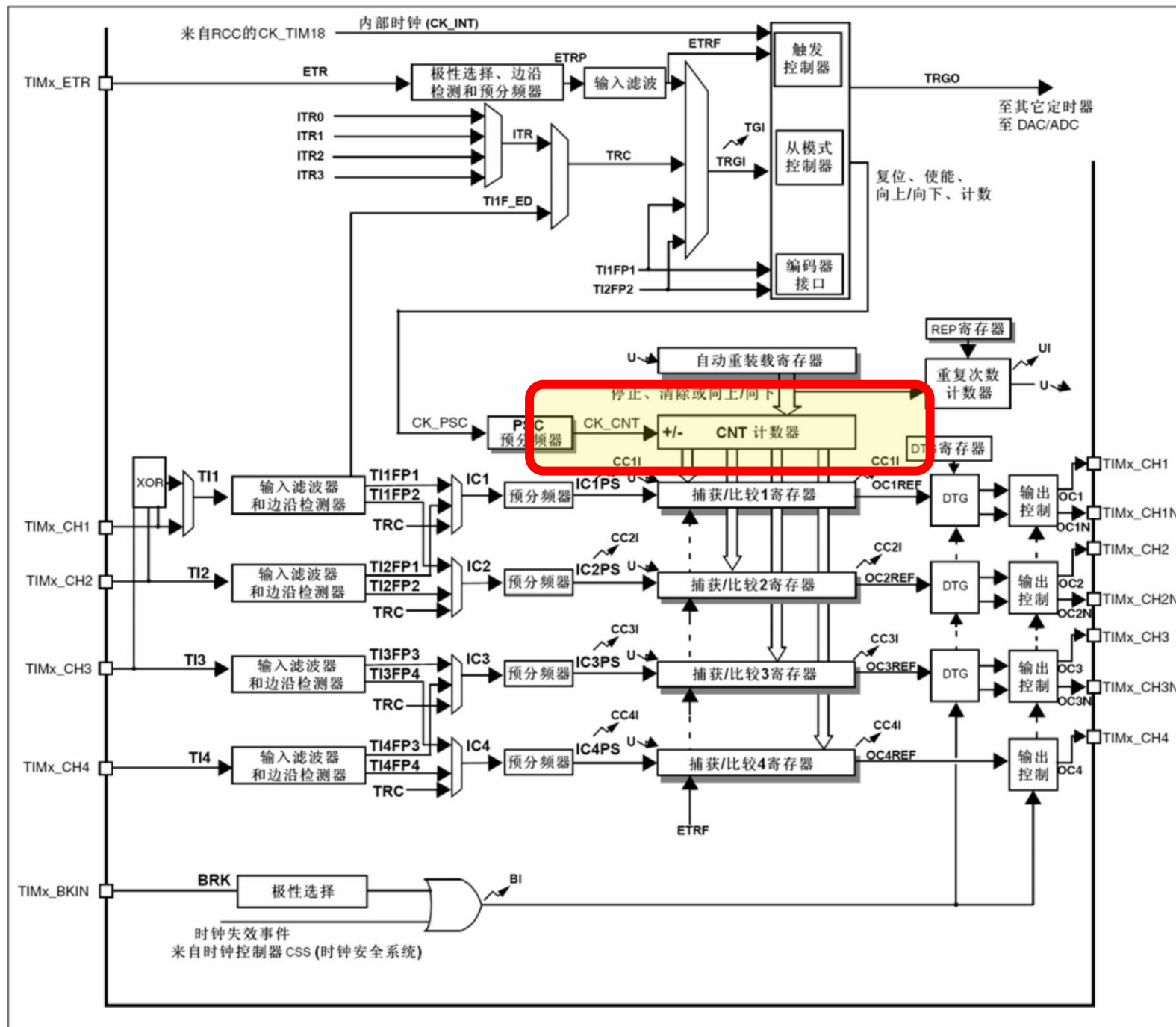








<https://docs.feishu.cn/v/wiki/A749wx8T0ioqfgkzZKlc9poknUf/a1>



对比项	① 内部时钟 CK_INT	②内部触发输入 ITRx	③ 外部时钟模式1 (定时器输入通道x) TIx	④ 外部时钟模式2(外部触发输入) ETR
来源	RCC分配	其他定时器通过内部连线送来的触发信号	外部引脚上的 CH1/CH2 输入信号	外部引脚上的 ETR 信号
来自	芯片内部	芯片内部	芯片外部	芯片外部
使用的引脚/通道	不占 GPIO	不占GPIO	占用定时器通道输入脚 TI1/TI2	占用专用 TIMx_ETR 引脚
本质作用	定时器自己的基础工作时钟	用别的定时器触发/驱动/同步本定时器	用外部脉冲驱动计数器	用外部脉冲驱动计数器
默认时钟源?	是	否, 需要主从模式配置	否, 需要专门配置	否, 需要专门配置
能独立工作?	能	不能, 依赖其他定时器输出	能, 只要外部有脉冲	能, 只要外部有脉冲
频率特性	高且稳定, 如 72MHz	取决于上游定时器输出节拍	取决于外部输入脉冲频率	取决于外部输入脉冲频率

对比项	① 内部时钟 CK_INT	②内部触发输入 ITRx	③ 外部时钟模式1 (定时器输入通道x) TIx	④ 外部时钟模式2 (外部触发输入) ETR
典型内部路径	RCC → TIMxCLK → PSC → CNT	其他TIM的TRGO → ITRx → 从模式控制器 → CNT/控制逻辑	TI1/TI2 → TRGI/从模式控制器 → CNT	ETR → ETR路径 → CNT
典型用途	定时中断、PWM、输出比较、输入捕获计时基准	一个定时器驱动另一个定时器、主从同步、扩展分频	传感器脉冲计数、编码器某一路计数、外部方波计数	专用外部时钟输入、外部事件计数
配置复杂度	最低	较高	中等	中等
你可以把它理解成	“自己带的发动机”	“别的定时器在内部推着你走”	“从 CH1/CH2 门口进来的外部节拍”	“从 ETR 专用门口进来的外部节拍”

核心计时单元：

- **PSC: Prescaler, 预分频器, 确定时间基准**

对输入时钟进行分频，降低计数频率，实现不同定时精度。

例：72MHz系统时钟经7200分频 → 10kHz计数时钟（每0.1ms计数一次）。

- **CNT: Counter, 计数器**

核心计数单元，根据时钟源进行递增 / 递减计数。

- **ARR: Auto-Reload Register, 自动重载寄存器**

计数器的最大值，计数到该值时自动复位。

定时器的溢出周期：

$$T = (PSC+1)(ARR+1)/TIMxCLK$$

如： 想要设置定时器的周期为50ms， 也就是 $T=0.05$

$$0.05 = (PSC+1)(ARR+1)/72000000$$

$$0.05 = ((7199+1)*(499+1))/72000000$$

$$0.05 = ((3599+1)*(999+1))/72000000$$

如果是1s呢？ 那就是

$$1 = (PSC+1)(ARR+1)/72000000$$

$$1 = ((7199+1)*(9999+1))/72000000$$

捕获 / 比较通道

- CCR: Capture/Compare Register, 捕获 / 比较寄存器

捕获模式：捕获外部信号边沿时的计数器值，用于测量频率 / 周期。

比较模式：与计数器值比较，匹配时产生中断或控制输出。

- CH1-CH4: Channel 1-4, 通道 1-4

4 个独立的输入 / 输出通道，用于连接外部引脚。

	输入捕获	输出比较
作用	测外部信号	产生输出信号
主动方	外部输入	定时器内部
关键寄存器用途	CCRx存捕获值	CCRx存比较值
典型应用	测频率、测脉宽	PWM、定时翻转

高级定时器TIM1特有功能

- DTG: Dead-Time Generator, 死区时间生成器

在互补输出通道之间插入延迟，防止上下桥臂短路。

- BKIN: Break Input, 刹车输入

外部紧急停止信号，立即关闭所有输出通道。是高级定时器中的一个安全保护输入信号。

- 重复次数寄存器 (REP)

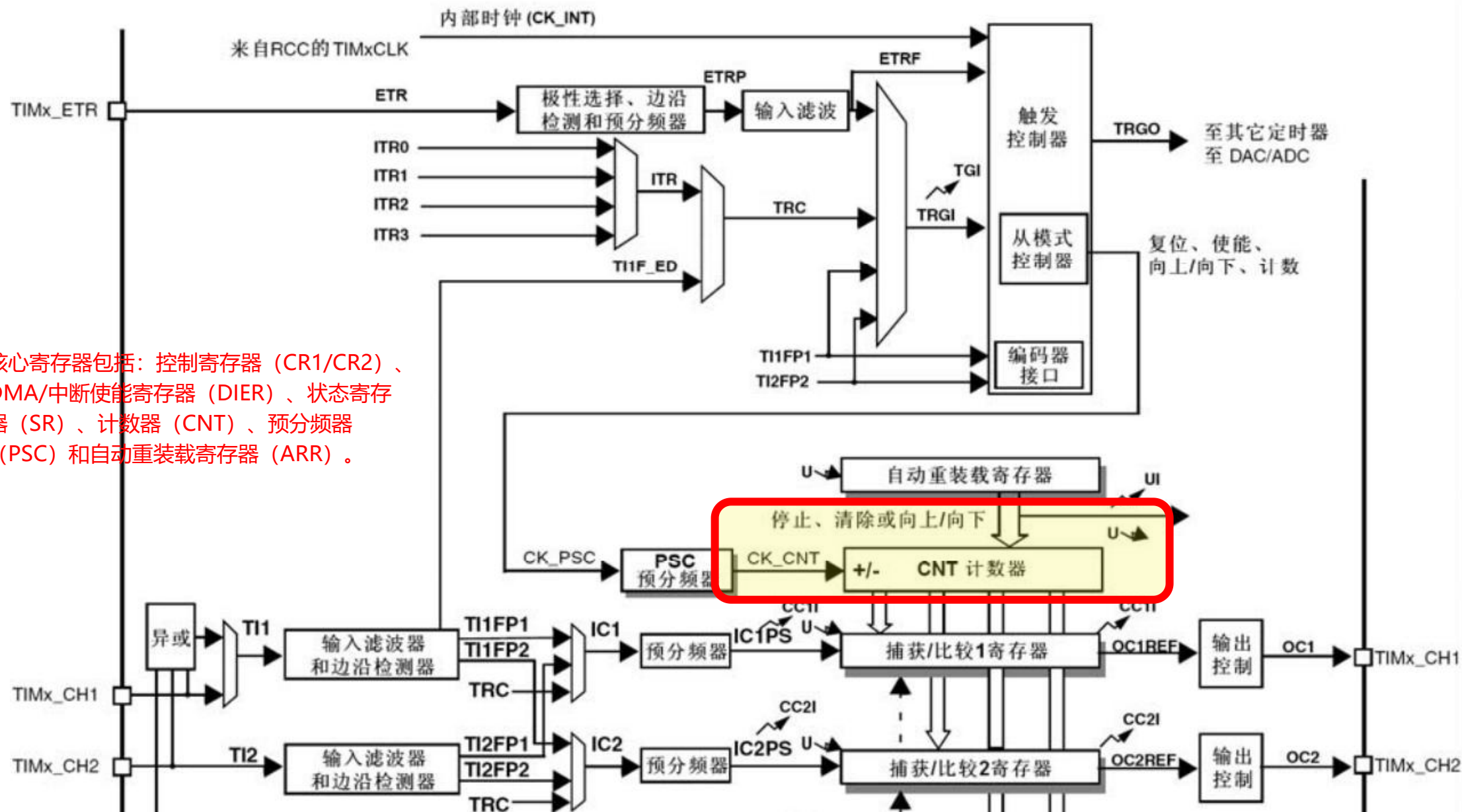
- 互补输出通道

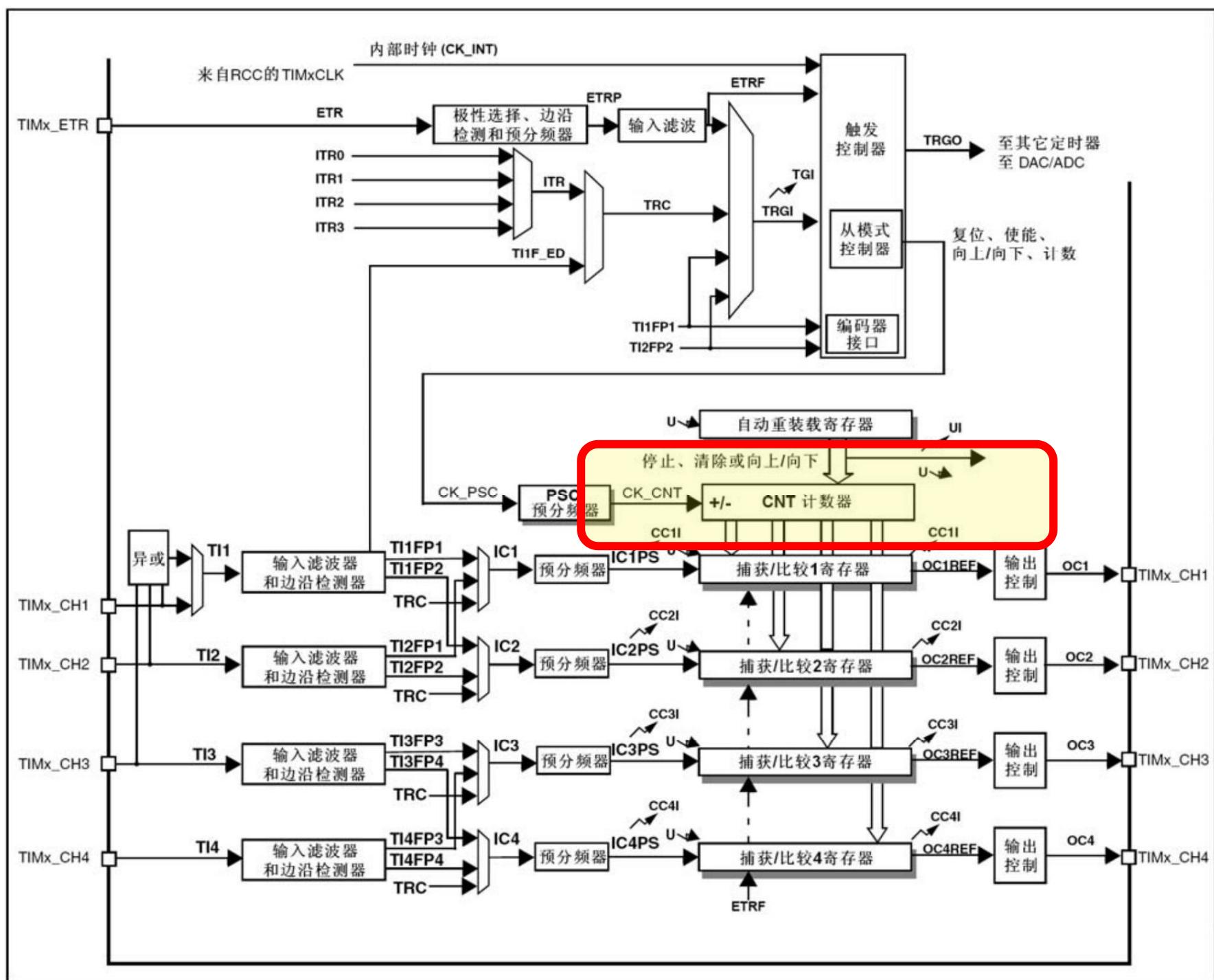
缩写	英文全称	常见中文译法
TIx	Timer Input x	定时器输入通道 x
TI1FP1	Timer Input 1 Filtered signal	TI1滤波后的输入信号
ETR	External Trigger	外部触发输入
TRGI	Trigger Input	触发输入
TRGO	Trigger Output	触发输出
ITRx	Internal Trigger x	内部触发 x
CHx	Channel x	通道 x
ICx	Input Capture x	输入捕获 x
OCx	Output Compare x	输出比较 x
OCxREF	Output Compare x Reference	输出比较参考信号

缩写	英文全称	常见中文译法
CCR_x	Capture/Compare Register x	捕获/比较寄存器 x
CNT	Counter	计数器
PSC	Prescaler	预分频器
ARR	Auto-Reload Register	自动重装寄存器
CK_CNT	Counter Clock	计数器时钟
BKIN	Brake Input	刹车/制动输入
CH_xN	Channel x Negative / Complementary Output	通道 x 互补输出
DTG	Dead-Time Generator	死区发生器
PWM	Pulse Width Modulation	脉宽调制

三、普通定时器TIM2、TIM3、TIM4、

核心寄存器包括：控制寄存器（CR1/CR2）、DMA/中断使能寄存器（DIER）、状态寄存器（SR）、计数器（CNT）、预分频器（PSC）和自动重装载寄存器（ARR）。





应用场景：

功能	场景	说明
定时计数	定时开关灯	周期性任务控制
DAC/ADC时钟	音频播放与数据采集	DAC、ADC的时钟同步
PWM输出	直流电机调速	通过占空比控制平均电压
输入捕获	示波器测量	精确测量脉冲宽度
输出比较	步进电机控制	生成精确时间间隔的脉冲
带死区的互补输出	无刷电机驱动	防止H桥短路，确保安全

TIM时钟初始化

- STM32单片机内，片内资源非常多，如GPIO、TIM、串行通信单元、RTC单元、SPI单元等等。但是是一个具体的应用，可能只用其中的部分资源。
- 为了**降低功耗**，不使用的单元，可以不给该单元时钟。
- 单片机复位后，所有的外设都不给时钟。在该应用中，需要给使用的外设时钟。
- STM32单片机时钟总线，有两条，所有的外设分别挂在这两条时钟线上。

```
//配置系统时钟,使能各外设时钟
void RCC_Configuration(void)
{
    SystemInit();    //系统时钟初始化
    RCC_APB2PeriphClockCmd(
        RCC_APB2Periph_GPIOA|RCC_APB2Periph_GPIOB |
        RCC_APB2Periph_GPIOC|RCC_APB2Periph_GPIOD , ENABLE ) ;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE);
}
```

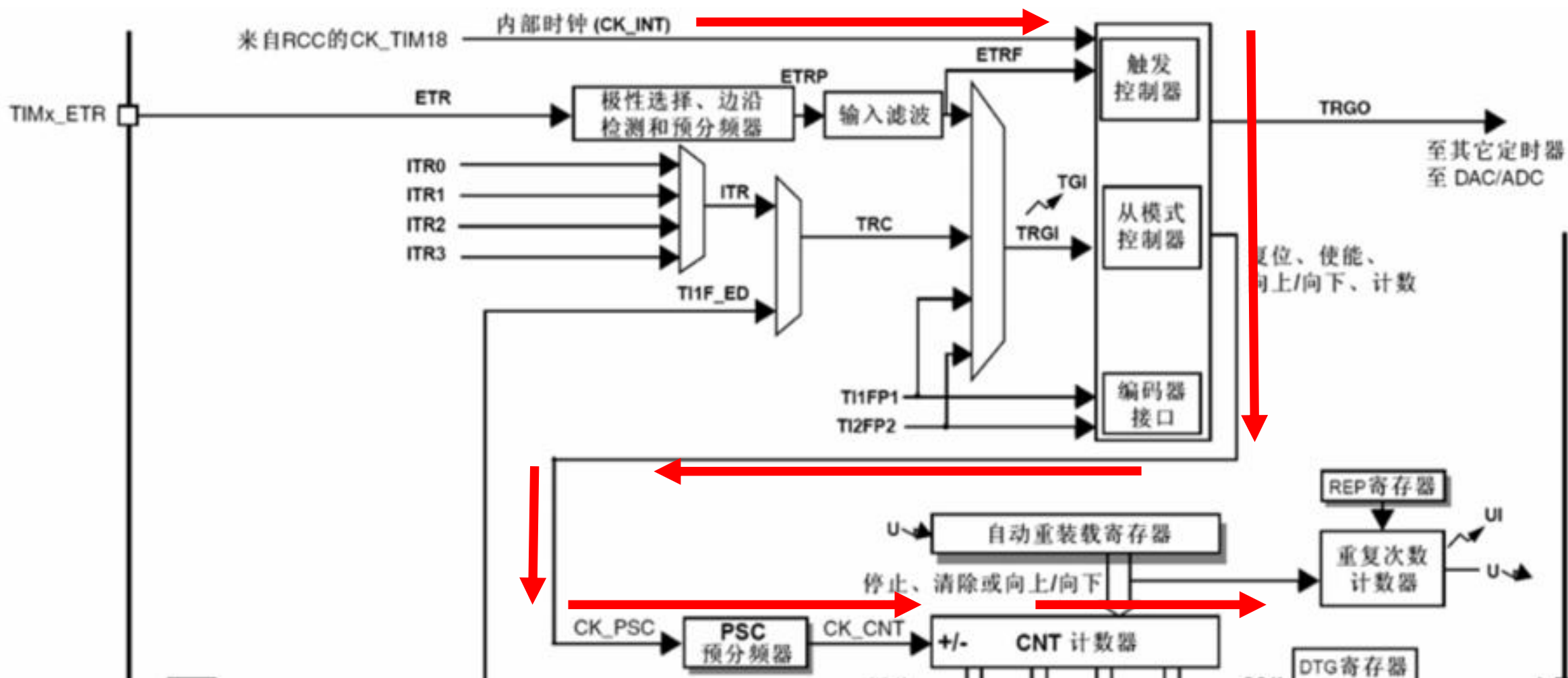
TIM分频初始化

- TIM初始化
- 1、TIM1的重复次数计数器，次数满后中断，其范围为1~256次。
- 2、TIMx的预分频系数，其范围为1~65536。
- 3、TIMx的周期，其范围为1~65536。
- 4、时钟分割、加减计数模式。
- 5、以上五条，使用函数 **TIM_TimeBaseInit**，进行设置。
- 6、设置时钟源，不设置的话，时钟来自内核时钟，72M。
- 7、使能TIMx。
- 8、如果时钟来自引脚，还可以对引脚上的时钟，进行预分频设置、滤波设置等。
- 9、是否允许溢出中断设置。

TIM 中断优先级初始化和中断函数

- TIM的中断优先级设置。
- TIM的溢出中断函数编写。
- 判断中断源，清中断标志。
- 写中断服务函数。

1、内部时钟源CK_INT下的TIM的溢出中断应用



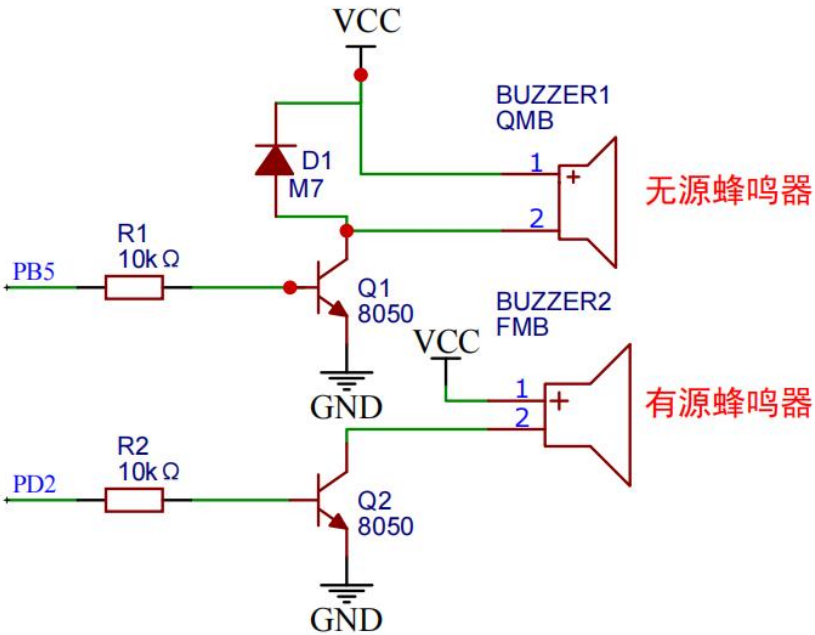
例程：一个带提示音和按键控制的多功能秒表系统

(通过 TIM1的1秒中断完成秒表计数；

通过 TIM2的1ms中断完成数码管动态扫描、按键检测、蜂鸣器控

制和 LED 流水灯。 (使用TIM的CNT溢出中断))

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2 ;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOD, &GPIO_InitStructure);
```



项目	有源蜂鸣器	无源蜂鸣器
内部是否带振荡电路	有（一般体积相对大些）	没有
是否可直接通电发声	可以	不能持续正常发声
控制方式	直接开关控制	需要频率信号驱动
是否可变音调	一般不行	可以
编程难度	简单	较高
典型用途	报警、提示	音乐、不同音调控制

- **TimeBaseInitTypeDef** 类型的结构体变量，用来保存 TIM 定时器的基础配置参数。

```
void TIM_Configuration(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;

    TIM_TimeBaseStructure.TIM_RepetitionCounter = 100-1;
    TIM_TimeBaseStructure.TIM_Prescaler=720-1;
    TIM_TimeBaseStructure.TIM_Period=1000-1;
    TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM1, & TIM_TimeBaseStructure);
    TIM_Cmd(TIM1, ENABLE);
    TIM_ITConfig(TIM1,TIM_IT_Update,ENABLE);
}
```

- `TIM_Prescaler=720-1;` 预分频值 719。
- 定时器计数频率约为： $72\text{MHz} / 720 = 100\text{kHz}$,也就是每计一次数约 $10\mu\text{s}$ （时间基准）。
- `TIM_Period=1000-1;` 自动重装值 `ARR = 999`
- 每数 1000 次溢出一次： $1000 \times 10\mu\text{s} = 10\text{ms}$
- 最终 TIM1 更新中断周期约是： $10\text{ms} \times 100 = 1\text{s}$, 所以 TIM1 实际上是 1秒中断一次。这和后面代码里秒表计时对应。
- `TIM_ClockDivision=TIM_CKD_DIV1;` 时钟分频不再额外分频。
- `TIM_CounterMode=TIM_CounterMode_Up;` 向上计数。
- `TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);`把配置的参数写入 TIM1寄存器。

```
TIM_Cmd(TIM1, ENABLE);
```

启动 TIM1 定时器。配置好之后，必须把定时器打开，它才会开始计数。

```
TIM_ITConfig(TIM1,TIM_IT_Update,ENABLE);
```

使能TIM1更新中断，当 TIM1 计数到 Period 并发生更新事件时：
产生中断请求，进入 TIM1_IRQHandler() 中断服务函数

注意：

★EXTI需要先告诉芯片“到底哪一个引脚接到哪一条中断线”，外部中断 EXTI 中断来源是GPIO引脚上的电平变化。

★TIM的中断源本来就在定时器内部，不需要再做“引脚到中断线”的映射配置。定时器中断来源是定时器模块内部事件，比如：
计数器更新事件（溢出）、比较匹配事件、捕获事件、触发事件
这些事件都是 TIM 外设自己产生的。

★所以外中断要写 EXTI_Configuration()，而定时器一般不用配这种函数。

GPIO引脚变化



AFIO选择这个引脚接到哪条EXTI线



EXTI模块判断触发条件



NVIC响应中断

定时器计数



发生更新/比较/捕获事件



TIM内部置中断标志位



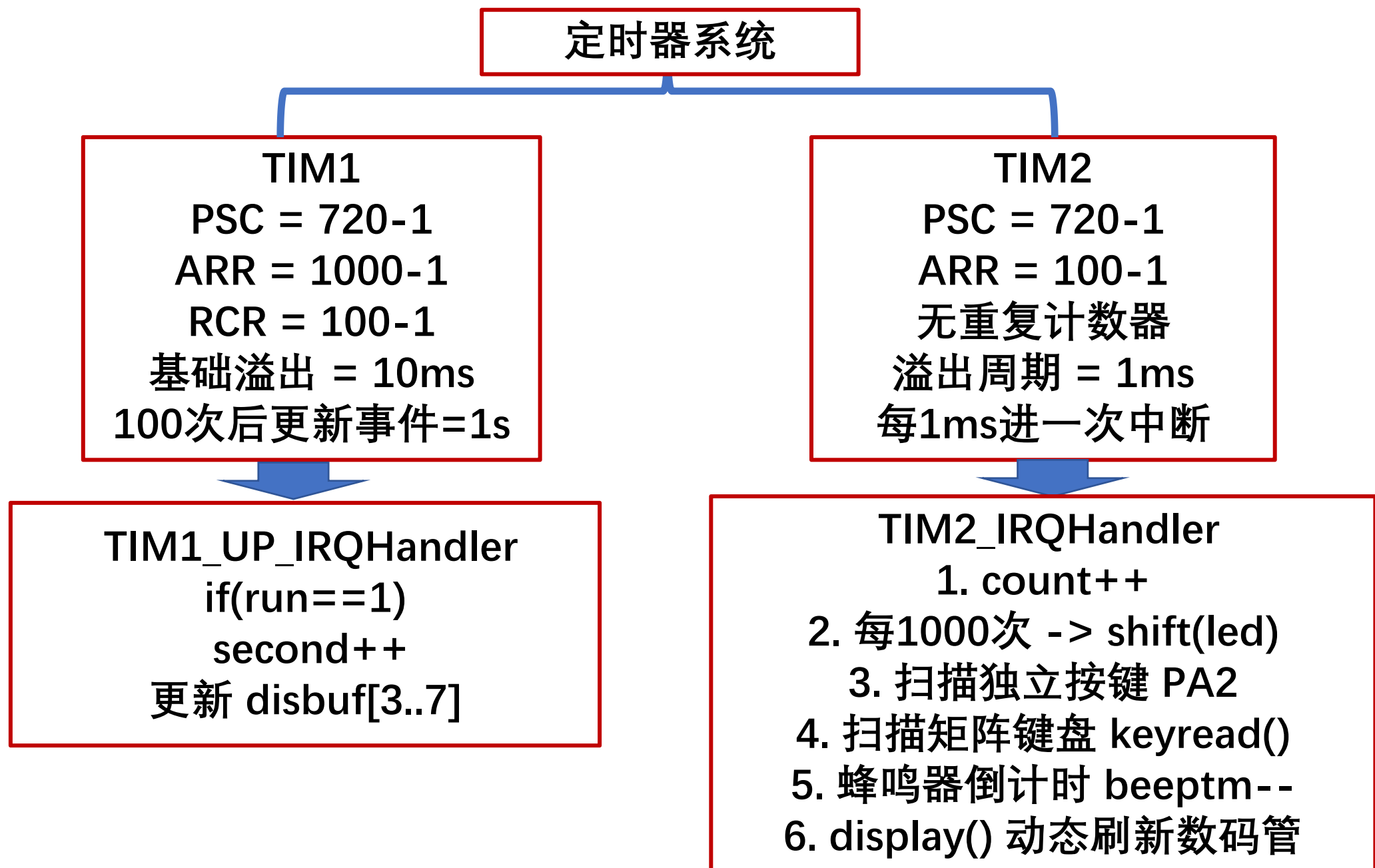
NVIC响应中断

配 TIM2定时器时基:

```
TIM_TimeBaseStructure.TIM_Prescaler=720-1;  
TIM_TimeBaseStructure.TIM_Period=100-1;  
TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;  
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;  
TIM_TimeBaseInit(TIM2, & TIM_TimeBaseStructure);  
TIM_Cmd(TIM2, ENABLE);  
TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);
```

- **TIM_Prescaler=720-1;** CNT每计一次数约 10us。
- **TIM_Period=100-1;** 自动重装值 ARR = 99
- 每数 100 次溢出一次: $100 \times 10\mu s = 1ms$
- **最终 TIM2 更新中断周期约是: 1ms**, 所以 TIM2 实际上是 1ms 中断一次。这和后面代码里流水灯1000ms, 扫描按键PA2速度、显示、蜂鸣器。。。对应。

TIM1 + TIM2 双中断调度关系图:



本程序采用TIM1和TIM2**双定时器中断调度**方式实现**多任务并行处理**

★TIM1配置为1s周期更新中断，主要负责秒表计数及显示缓冲区更新；

★TIM2配置为1ms周期更新中断，主要负责数码管动态扫描、独立按键检测、矩阵键盘扫描、蜂鸣器定时控制以及 LED 流水灯节拍累加。

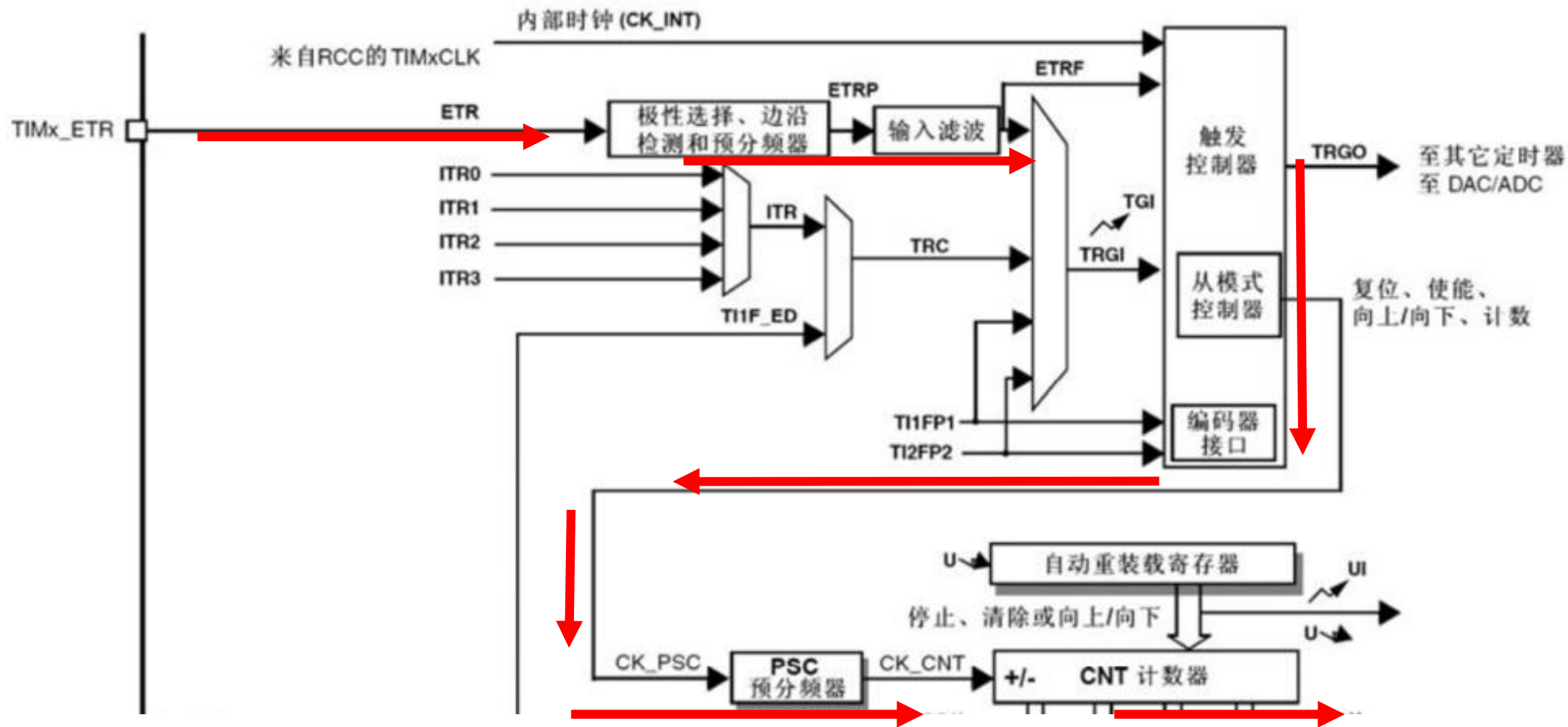
★主程序初始化完成后进入空循环，前台无任务处理，**所有功能均在后台中断服务程序中完成。**

★由于TIM1的中断优先级高于TIM2，因此在两个中断同时到来时，TIM1可抢占TIM2，从而**保证秒计数任务的实时性。**

练习：

- 修改蜂鸣器秒叫时长为2s。
- 修改流水灯间隔时间为2s
- 更换矩阵按键控制清零。

2、外部引脚脉冲源ETR下的TIM的计数模式应用



2、外部引脚脉冲源ETR下的TIM的计数模式应用

- 计数模式
- 时钟源自**引脚**
- TIM1: PA12 接到数码管上了，不能用
- TIM2: PA0 可以用
- TIM3: PD2 接蜂鸣器了，不能用
- TIM4: 无
- **计数模式下，一般不允许溢出中断。**

TIM时钟初始化

```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);  
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
```

TIM分频初始化

```
TIM_TimeBaseStructure.TIM_Prescaler=0;
TIM_TimeBaseStructure.TIM_Period=10;
    //ARR=10,即CNT溢出周期10个脉冲
TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;
TIM_TimeBaseInit(TIM2, & TIM_TimeBaseStructure);
TIM_Cmd(TIM2, ENABLE);
//没有使能中断，因为没有用到TIM2溢出中断，所以也不用设置NVIC，
也不用写TIM2中断函数。

TIM_ETRClockMode1Config(TIM2,TIM_ExtTRGPSC_OFF,
    TIM_ExtTRGPolarity_NonInverted, 0x0);
```

把 TIM2 配置为**外部时钟模式1**，即使用 ETR 外部触发输入作为计数时钟来源。

```
TIM_ETRClockMode1Config(TIM2,TIM_ExtTRGPSC_OFF,  
TIM_ExtTRGPolarity_NonInverted, 0x0);
```

参数含义：

- ★TIM_ExtTRGPSC_OFF外部触发预分频关闭，不分频。
- ★TIM_ExtTRGPolarity_NonInverted外部脉冲极性不反转。
- ★0x0 不加数字滤波。

所以 TIM2 的计数不是靠内部时钟，而是每来一个外部脉冲，TIM2 计数器加 1。

polarity/pə'lærəti/

在 STM32F10x 芯片手册/数据手册 中，PA0 的复用功能之一就是：

PA0 -> TIM2_ETR

即：PA0 本身默认复用为 TIM2 的 ETR 输入，这是芯片内部硬件**固定映射**。
所以代码里只要：**把 PA0 配成输入**，再把 TIM2 配成 ETR 外部时钟模式，
那么 PA0 上的脉冲就能送给 TIM2 作为计数时钟。

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING ;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

PA0 配成浮空输入，作为 TIM2 的外部时钟输入引脚。

例程：计数器

用 TIM2、TIM3 分别作为外部脉冲计数器；
两个计数值显示在 8 位数码管上

- ★左 4 位显示 TIM2

- ★右 4 位显示 TIM3

在 SysTick 1ms 中断里读取两个计数器的当前值，显示到数码管上；
再通过矩阵按键实现两个计数器的启动、停止、清零。

★矩阵按键控制

键 1：启动 TIM2

键 2：停止 TIM2

键 3：TIM2 清零

键 4：启动 TIM3

键 5：停止 TIM3

键 6：TIM3 清零

•TIM3_ETR 常对应 **PD2**

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING ;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
GPIO_Init(GPIOD, &GPIO_InitStructure);
```

同样，TIM3 配置为外部时钟模式1，即使用 ETR 外部触发输入作为计数时钟来源。

```
TIM_ETRClockMode1Config(TIM3,TIM_ExtTRGPSC_OFF,TIM_ExtTRGPolarity_NonInverted, 0x0);
```

参数含义：

- ★TIM_ExtTRGPSC_OFF外部触发预分频关闭，不分频。
- ★TIM_ExtTRGPolarity_NonInverted外部脉冲极性不反转。
- ★0x0 不加数字滤波。

所以 TIM3 的计数不是靠内部时钟，而是每来一个外部脉冲，TIM3 计数器加 1。

polarity/pə'laerəti/

```
count1=TIM_GetCounter(TIM2);    //获取TIM2计数器CNT中的值  
count1=TIM_GetCounter(TIM3);
```

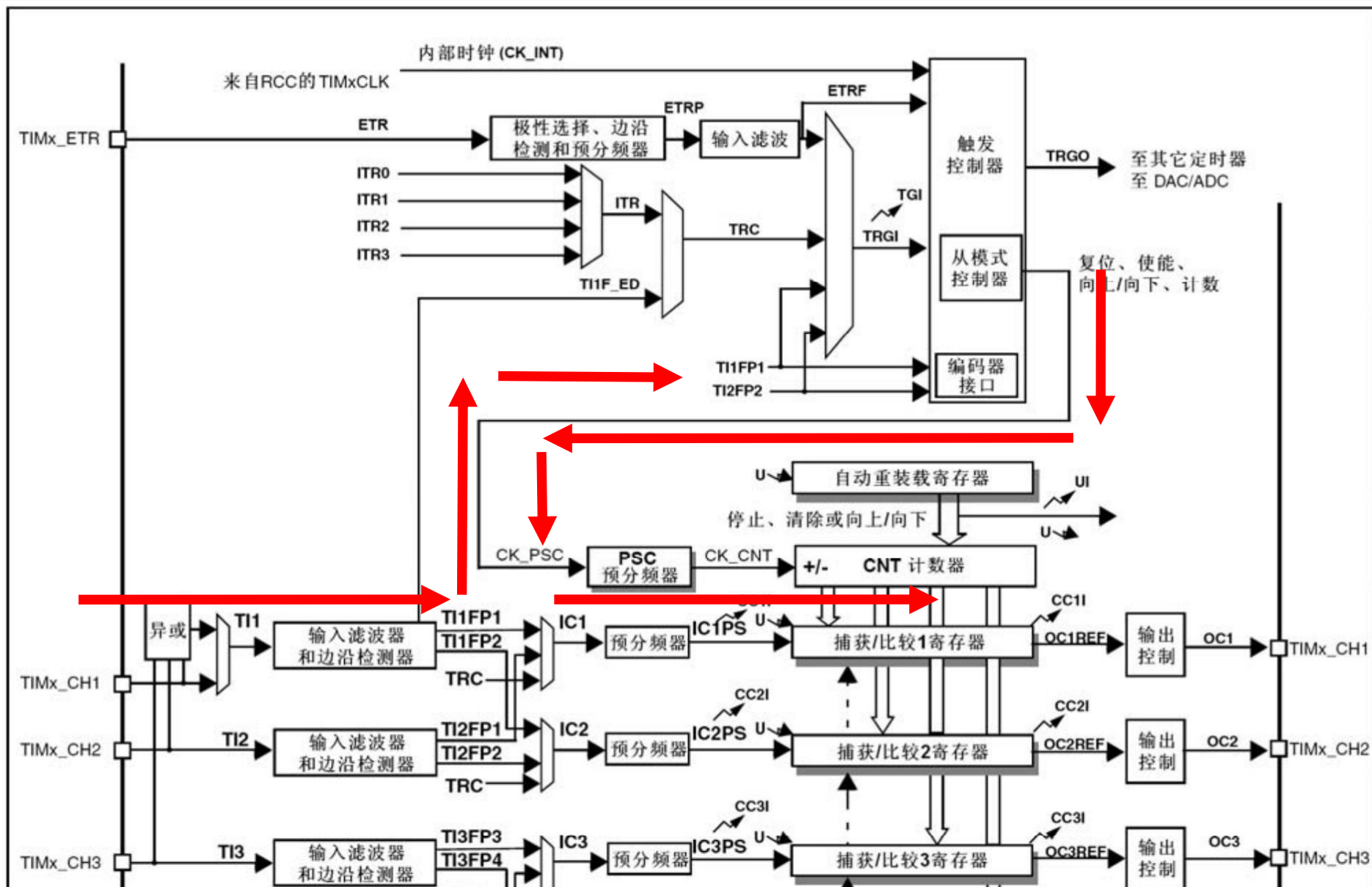
```
TIM_SetCounter(TIM2,0);    //将TIM2计数器CNT中的值清0  
TIM_SetCounter(TIM3,0);
```

```
TIM_Cmd(TIM2, ENABLE);  
TIM_Cmd(TIM2, DISABLE);
```

课堂练习：

- 改变TIM3计数周期，如从65535换为20（**超过20个脉冲，超出计数范围**），观察到什么现象？
- 把up换成down
- 把流水灯间隔亮改为0.2s，改变TIM2或者TIM3的预分频，观察计数器CNT里的值的变化，几个脉冲加1？

3、通道引脚信号下的TIM的计数模式应用



```
TIM_ TIxExternalClockConfig(TIM4, TIM_ TIxExternalCLK1Source_TI1,  
TIM_ICPolarity_Rising, 0);
```

- ★ TIx: Timer Input x, 定时器输入通道 x, 例如 TI1、TI2
- ★ 把 TIM4 配置成外部时钟模式, 用通道 1 输入引脚上的外部脉冲作为计数时钟。
- ★ TIM4_CH1 引脚上每来一个有效上升沿, TIM4 的计数器 CNT 加 1。
- ★ **注意:** 用通道输入做计数器, 只能使用1、2通道, 其它通道不可以。

例程：使用TIM实现通道引脚信号计数，计数信号可采用流水灯。（TIM4的CH1默认的引脚是PB6）。

练习：改变引脚信号频率，观察计数结果。

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7 | GPIO_Pin_8 | GPIO_Pin_9 ;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

例程：基于 STM32F103RBT6 的可调电子时钟与流水灯控制系统（多定时器中断应用）

用 TIM1 1ms 中断刷新数码管显示

用 TIM2 1ms周期扫描按键，调时/调分/调秒

用 TIM3 1s控制流水灯

用 TIM4 1s 计时

★矩阵按键控制

键 1：调时+1

键 2：调分+1

键 3：调秒+1

键 4：调时-1

键 5：调分-1

键 6：调秒-1

作业：

- 利用系统滴答定时器增加整点蜂鸣器报时功能
- 要求：几点响几下，如01:00:00嘀一下、12:00:00滴12下，13:00:00-24:00:00，嘀1-12下